

BookMyShow Platform System Design

Comprehensive Technical Design Explanation: Object-Oriented Low-Level Design (LLD), Scalable Distributed High-Level Architecture (HLD), and Integrated System Flow.

Executive Summary

BookMyShow is a mission-critical entertainment ticketing platform serving millions of concurrent moviegoers. The system faces extreme concurrency spikes during major blockbuster ticket drops, requiring a robust architecture that guarantees **zero double bookings**, sub-second search latency, resilient payment execution, and decoupled notification fulfillment. This document provides the complete end-to-end technical explanation across object-oriented principles, distributed microservices, data management, and operational recovery.

Part 1: Low-Level Design (LLD)

The Low-Level Design defines the object-oriented structure, class invariants, and behavioral patterns governing the booking lifecycle.

1. Core Domain Entities & Relationships

- **Movie:** Contains film metadata including title, duration in minutes, genre, language, and certificate rating.
- **Theater:** Represents physical cinema locations with name, city, and composite collections of screens.
- **Screen:** Auditorium containing structural rows and seats, implementing the SeatComponent composite contract.
- **Seat:** Physical seat model tracking row, number, tier (STANDARD, PREMIUM, RECLINER, IMAX), base price, and delegating to SeatState.
- **Show:** Scheduled screening of a Movie in a Screen at specific start and end timestamps.
- **User:** Customer entity managing profile credentials, booking history, and wallet balances.
- **Booking:** Transactional aggregate uniting User, Show, reserved Seats, issued Tickets, selected Snacks, total amount, and Payment reference.
- **Payment:** Financial transaction details with gateway transaction IDs, amount, and settlement status (PENDING, SUCCESS, FAILED).
- **Snack:** Concession items extensible via SnackDecorator for size upgrades and topping add-ons.

```

export class Seat implements SeatComponent {
  private id: string;
  private type: SeatType;
  private row: number;
  private number: number;
  private basePrice: number;
  private state: SeatState;

  constructor(id: string, type: SeatType, row: number, number: number, basePrice: number, state: SeatState) {
    this.id = id;
    this.type = type;
    this.row = row;
    this.number = number;
    this.basePrice = basePrice;
    this.state = state;
  }

  public getId(): string { return this.id; }
  public getType(): SeatType { return this.type; }
  public getRow(): number { return this.row; }
  public getNumber(): number { return this.number; }
  public getBasePrice(): number { return this.basePrice; }
  public getState(): SeatState { return this.state; }
  public setState(state: SeatState): void { this.state = state; }

  public isAvailable(): boolean {
    return this.state.getStatus() === SeatStatus.AVAILABLE;
  }

  public reserve(): void { this.state.reserve(this); }
  public release(): void { this.state.release(this); }
  public confirm(): void { this.state.confirm(this); }
  public getAvailableCount(): number { return this.isAvailable() ? 1 : 0; }
  public getTotalCount(): number { return 1; }
}

```

2. Design Principles (SOLID)

- **Single Responsibility Principle (SRP):** Seat solely tracks seating coordinates and state transitions; PaymentHandler classes isolate individual checkout stages; Logger handles thread-safe audit trails.
- **Open/Closed Principle (OCP):** The system extends functionality without modifying tested core logic. Pricing algorithms implement PricingStrategy, and concessions implement SnackDecorator.
- **Liskov Substitution Principle (LSP):** All ticket types (StandardTicket, PremiumTicket, IMAXTicket) substitute Ticket without breaking client expectations.
- **Interface Segregation Principle (ISP):** Narrow interfaces (ISnack, SeatComponent, Command) avoid forcing unused methods onto clients.
- **Dependency Inversion Principle (DIP):** BookingWorkflow depends strictly on abstractions injected via constructor (SeatAllocationStrategy, PaymentGatewayStrategy, BookingSubject).

```

export interface PricingStrategy {
  calculatePrice(basePrice: number, show: Show): number;
}

export class WeekendSurgePricingStrategy implements PricingStrategy {
  public calculatePrice(basePrice: number, show: Show): number {
    return basePrice * 1.25;
  }
}

export abstract class BookingWorkflow {
  protected seatAllocator: SeatAllocationStrategy;
  protected paymentGateway: PaymentGatewayStrategy;
  protected eventBus: BookingSubject;

  constructor(allocator: SeatAllocationStrategy, gateway: PaymentGatewayStrategy, bus: BookingSubject) {
    this.seatAllocator = allocator;
    this.paymentGateway = gateway;
    this.eventBus = bus;
  }

  public abstract processBooking(request: BookingRequest): Booking;
}

```

3. Design Patterns Integration

- **Simple Factory (TicketFactory):** Polymorphically instantiates ticket variants based on seat tier and format.
- **Builder (BookingBuilder):** Incrementally constructs complex Booking aggregates while enforcing domain invariants.
- **Singleton (Logger):** Guarantees a single centralized instance for thread-safe system audit logging.
- **Decorator (SnackDecorator):** Dynamically layers toppings and size upgrades onto base snacks at runtime.
- **Composite (SeatComponent):** Unifies Screen, Row, and Seat into a hierarchy for uniform occupancy counts.
- **Strategy (PricingStrategy, SeatAllocationStrategy):** Encapsulates interchangeable pricing and seating selection logic.
- **Observer (BookingSubject):** Publishes booking confirmation events to email, SMS, and push notification listeners.
- **Command (BookingInvoker):** Encapsulates actions into commands with an undo() method, enabling clean LIFO rollbacks.
- **State (SeatState):** Governs valid seat transitions (AVAILABLE → HELD → BOOKED) and throws exceptions on invalid moves.
- **Template Method (BookingWorkflow):** Enforces a standard 5-step booking skeleton while letting subclasses override validation.
- **Chain of Responsibility (PaymentHandler):** Chains coupon validation, wallet balance deductions, and tax computations.

Part 2: High-Level Design (HLD)

The High-Level Design establishes the scalable, fault-tolerant distributed system architecture that powers the platform under heavy loads.

1. Layered Architecture & Microservices

- **Edge & Ingress:** Cloudflare CDN caches static assets (posters, trailers); Application Load Balancer terminates TLS 1.3.
- **API Gateway:** Enforces JWT authentication, Token Bucket rate limiting, and Idempotency-Key deduplication.
- **BookingService:** Manages seat locks via Redis Lua scripts, coordinates checkout, and initiates database transactions.
- **PaymentService:** Interfaces with external PSPs (Razorpay/Stripe) via Resilience4j Circuit Breakers with idempotent retries.
- **SearchService:** Elasticsearch cluster powering geospatial radius search and multi-attribute movie filtering.
- **CatalogService:** Serves high-volume read queries for movie schedules and theater layouts backed by Redis read-through caches.
- **NotificationService:** Asynchronous worker pool consuming Kafka events to deliver QR tickets via email and SMS.

2. Data Management, Sharding & Caching

City-Based Sharding Strategy: Over 99% of moviegoer queries are localized to a specific metropolitan area. Horizontally sharding PostgreSQL by city_id guarantees that all related records (theaters, screens, shows, seats, bookings) reside on the same physical database node, eliminating cross-shard joins and distributed two-phase commits.

Two-Tier Caching Architecture: Movie metadata is cached for 300 seconds (Cache-Aside). Live seat availability maps use a short 10-second TTL. Seat reservations use atomic Redis distributed locks (SETNX) with an 8-minute TTL, eliminating row lock contention on the database.

```
CREATE TABLE shows (  
  id VARCHAR(36) PRIMARY KEY,  
  movie_id VARCHAR(36) NOT NULL REFERENCES movies(id),  
  screen_id VARCHAR(36) NOT NULL REFERENCES screens(id),  
  start_time TIMESTAMP WITH TIME ZONE NOT NULL,  
  end_time TIMESTAMP WITH TIME ZONE NOT NULL,  
  base_ticket_price DECIMAL(10,2) NOT NULL  
);  
  
CREATE TABLE show_seats (  
  id VARCHAR(36) PRIMARY KEY,  
  show_id VARCHAR(36) NOT NULL REFERENCES shows(id),  
  seat_id VARCHAR(36) NOT NULL REFERENCES seats(id),  
  status VARCHAR(20) NOT NULL,  
  locked_until TIMESTAMP WITH TIME ZONE,  
  CONSTRAINT uq_show_seat UNIQUE (show_id, seat_id)  
);  
  
CREATE TABLE bookings (  
  id VARCHAR(36) PRIMARY KEY,  
  user_id VARCHAR(36) NOT NULL,  
  show_id VARCHAR(36) NOT NULL REFERENCES shows(id),  
  total_amount DECIMAL(10,2) NOT NULL,  
  status VARCHAR(20) NOT NULL,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

3. CAP Theorem Decisions & Fault Tolerance

- **Seat Locking & Booking (CP):** In the event of a network partition, the system strictly prioritizes data consistency over availability to eliminate double bookings. Conflicting write requests are rejected.
- **Search & Catalog (AP):** Catalog browsing prioritizes 100% availability with eventual consistency. Brief replication delays in movie listings are acceptable.
- **Circuit Breakers:** Wrap third-party payment gateways. If gateway error rates exceed 30%, calls fail fast, preserving application thread pools.
- **Dead-Letter Queues (DLQ):** Unprocessable or failed Kafka messages route to DLQ topics after 3 exponential retries, preventing consumer lag.

4. Monitoring, Observability & SLOs

- **Latency SLO:** 99% of booking transactions complete in < 2.0 seconds (p99 < 2.0s).
- **Availability SLO:** 99.95% platform uptime across calendar months.
- **Observability Stack:** Prometheus metric collection, Grafana operational dashboards, OpenTelemetry distributed tracing with X-Correlation-ID, and PagerDuty alerting.

Part 3: Integrated Flow & DevOps Strategy

This section models the unified lifecycle across LLD domain objects and HLD distributed microservices, details failure compensation, and outlines DevOps deployment.

1. Unified Booking Sequence

1. **Discovery:** Client queries SearchService and selects showtime.
2. **Lock Acquisition:** Client submits seat selection; BookingService executes Redis Lua script (SETNX with 480s TTL). Returns lock_token.
3. **Concessions & Pricing:** Snacks wrapped via SnackDecorator; price compiled via PaymentHandler chain.
4. **Payment Settle:** BookingService calls PaymentService via gRPC; payment executed through Circuit Breaker to PSP.
5. **Persistence:** ACID commit to PostgreSQL city shard (inserts bookings, marks show_seats as BOOKED). Redis temporary lock keys deleted.
6. **Event Dispatch:** BookingService emits BookingConfirmedEvent to Kafka topic booking_events. NotificationService dispatches QR tickets via Email/SMS, and AnalyticsService ingests business metrics.

2. Failure & Recovery Handling

- **Payment Fails Mid-Transaction:** PSP returns 402 Declined. BookingService deletes Redis hold keys immediately, resetting seats to AVAILABLE. No database row is committed.
- **Seat Hold Expires (Timeout > 8 mins):** Redis TTL expires naturally. BookingService validates the lock token before checkout. If missing, it rejects with 409 Conflict. If the user was charged at the expiration boundary, an automated instant reversal refund is triggered.
- **Queue Consumer Crashes:** Kafka uses manual offset commits after message delivery. If a worker pod crashes, Kafka rebalances the consumer group and a sibling pod resumes from the last uncommitted offset without message loss.
- **Notification Delivery Outage:** Downstream email/SMS gateway 5xx errors trigger 3 exponential backoff retries. Exhausted messages move to a Dead-Letter Queue (DLQ) for alerting and replay. The core booking remains 100% confirmed.

3. Deployment & DevOps Notes

- **Containerized Kubernetes Stack:** Microservices deployed as Docker containers on Amazon EKS with Horizontal Pod Autoscaling (HPA) based on CPU (>70%) and RPS.
- **CI/CD Pipeline:** GitHub Actions runs linting, unit tests, container vulnerability scans (Trivy), and GitOps deployment via ArgoCD.
- **Secrets Management:** Credentials stored securely in AWS Secrets Manager / HashiCorp Vault and mounted at runtime.
- **Blue-Green Deployment:** New releases deployed to idle Green environment; canary traffic shifted (10% → 100%) through Application Load Balancer. Instant rollback (< 5 seconds) to Blue if error rates exceed 0.5%.